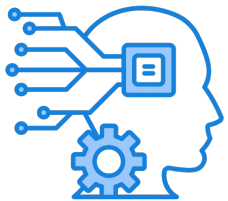
Universidade Federal de Goiás
Instituto de Informática
Prof. Dr. Ronaldo Martins da Costa

UFG
INF
INSTITUTO DE INFORMÁTICA



Aprendizado de Máquina
Um pouco mais sobre o Sklearn

- Esta biblioteca fornece uma seleção de ferramentas eficientes para aprendizado de máquina e modelagem estatística, incluindo:
  - Regressão
  - **Classificação**
  - Agrupamento e redução de dimensionalidade

- Criado em 2007 por Criação por David Cournapeau (https://scipy.org/)
- 2008 - 2009 Gaël Varoquaux liderou a reescrita e a expansão do projeto
- Lançado oficialmente em 2010

Entendendo e Preparando Dados com Scikit Learn

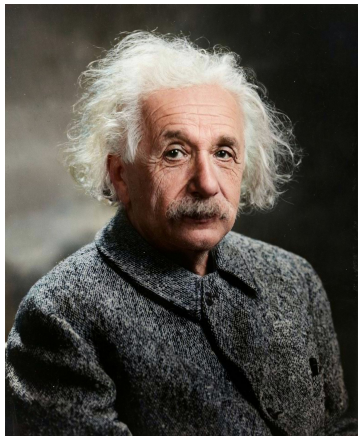
- **DataSet:** Suponhamos que o objetivo seja identificar possíveis clientes para minha empresa e que eu disponha de uma base de dados contendo informações cadastrais e comportamentais de 100 pessoas

- Como preparar estes dados para um processamento ?

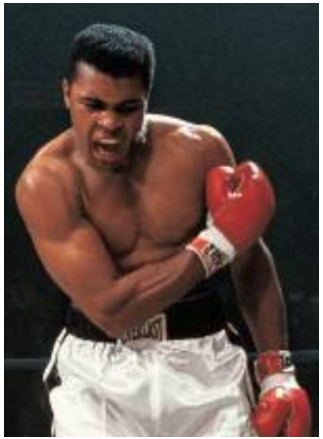


Aprendizado Supervisionado - O que é o conjunto X e o conjunto y ?

regressao_11







X

```
Cabelo, Olhos, Peso, Altura, …
Branco, Castanhos, 53, 1.52 …
Loiro, Azuis, 63, 1.78, …
Preto, Castanho Escuro, 98, 1.91,
…
```

y

```
Albert Einstein
Princesa Diane
Muhammad Ali
```

Entendendo o exemplo construído

regressao_11

```
pred = modelo.predict(X)
```

Faz uma previsão para cada amostra, geralmente recebendo apenas X como entrada

Sempre será executado depois que um fit for executado

Qual seria o resultado?

```
pred = modelo.predict(x)
print(pred)
```

sendo X:

```
Cabelo, Olhos, Peso, Altura, …
Loiro, Azuis, 63, 1.78, …
```

Entendendo o exemplo construído

regressao_11

```
pred = modelo.predict(X)
```

Faz uma previsão para cada amostra, geralmente recebendo apenas X como entrada

Sempre será executado depois que um fit for executado

Qual seria o resultado?

```
pred = modelo.predict(x)
print(pred)
```

sendo X:

```
Cabelo, Olhos, Peso, Altura, …
Loiro, Azuis, 63, 1.78, …
```



*As outras partes do código são para exibir tabelas e dados no template

Entendendo e Preparando Dados com Scikit Learn

- **DataSet**
  - Uma coleção de dados é chamada de conjunto de dados, é composta de:
    - **Features** (Características)
      - Elas também são conhecidas como:
        - Preditores,
        - Entradas, ou
        - Atributos.
    - **Response** (Resposta)
      - Ela também é conhecida como:
        - Destino,
        - Rótulo, Label ou
        - Saída.

Entendendo e Preparando Dados com Scikit Learn

- **DataSet:** Suponhamos que o objetivo seja identificar possíveis clientes para minha empresa e que eu disponha de uma base de dados contendo informações cadastrais e comportamentais de 100 pessoas

- Continuando com a preparação dos dados para o processamento…



Entendendo e Preparando Dados com Scikit Learn

- Caso eu utilize todos esses dados para identificar padrões de comportamento e características comuns entre os clientes existentes, poderei treinar um modelo de aprendizado de máquina capaz de analisar uma nova pessoa e classificá-la como um potencial cliente ou não, com base nos padrões aprendidos a partir dos dados



Entendendo e Preparando Dados com Scikit Learn

- Para avaliar corretamente o desempenho do modelo, é fundamental utilizar dados de pessoas que não fizeram parte do conjunto de treinamento. Se o modelo for testado com os mesmos dados utilizados para aprender os padrões, a avaliação ficará comprometida, pois estaremos medindo apenas a capacidade do modelo de memorizar os exemplos conhecidos, e não sua capacidade de generalização para novos casos

- Quanto tempo levará para conseguir os dados de uma nova pessoa ?

- Qual o custo para conseguir estes dados ?

- E quando não podemos aguardar até obter os dados de uma nova pessoa ?



Entendendo e Preparando Dados com Scikit Learn

E ainda…

➔ Utilizar todos os dados para treinar, validar e testar um modelo seria como fornecer aos alunos uma lista com todas as questões que cairão na prova e, posteriormente, avaliar seu conhecimento utilizando algumas dessas mesmas questões. Nesse caso, estaríamos medindo mais a capacidade de memorização do que o aprendizado real

➔ Da mesma forma, em Machine Learning, queremos que o modelo aprenda padrões nos dados de treinamento e seja capaz de realizar boas previsões em dados novos, nunca vistos anteriormente. Essa capacidade de aplicar o conhecimento adquirido em novas situações é chamada de **generalização**



Entendendo e Preparando Dados com Scikit Learn

➔ Solução: Separar os dados em 3 conjuntos

Treinamento
(entre 70% a 80% das amostras)



Material de Estudo

Validação
(entre 10% a 15% das amostras)



Exercícios de Preparação

Teste
(entre 10% a 15% das amostras)



Prova

Entendendo e Preparando Dados com Scikit Learn

- Conjunto de Treinamento (70% a 80% dos dados)
  - Objetivo: Ensinar o modelo a encontrar padrões nos dados
  - Exemplo:

Idade	Renda	Visitas Site	Comprou
25	3000	15	Sim
52	8000	2	Não
31	4500	12	Sim
...	...	...	...

- Pessoas com maior renda compram mais?
- Pessoas que visitam mais o site compram mais?
- Determinada faixa etária tem mais interesse em comprar?

A intenção é criar uma função:

Cliente = f(idade, renda, comportamento)



Entendendo e Preparando Dados com Scikit Learn

- Conjunto de Validação (10% a 15% dos dados)
  - São realizadas várias validações para ajustar qual o melhor modelo e parâmetros. Nesta fase são apresentados ao modelo dados que ele não utilizou no treinamento
  - Exemplo:

Idade	Renda	Visitas Site	Comprou
28	3800	10	Sim
45	7200	3	Não
36	5000	14	Sim

- Com estes novos dados, o modelo vai realizar as previsões e verificar se acertou na identificação se Comprou?
- O modelo então "armazena" as configurações deste teste



Entendendo e Preparando Dados com Scikit Learn

- ➔ Conjunto de Validação com "Validação Cruzada" - *Cross Validation*
  - ◆ Na validação cruzada, normalmente não é definido previamente um percentual fixo (como 10%, 15% ou 20%) para validação. O conjunto de dados é dividido em K subconjuntos (folds) de tamanho aproximadamente igual

  - ◆ O processo realiza k iterações, onde:
    - em cada iteração, K-1 subconjuntos são utilizados para treinamento;
    - o subconjunto restante é utilizado para validação;
    - o desempenho do modelo é avaliado e armazenado.



Entendendo e Preparando Dados com Scikit Learn

- Conjunto de Validação com "Validação Cruzada" - *Cross Validation*
  - Ao final das K iterações, calcula-se a média das métricas obtidas (por exemplo, acurácia, precisão, RMSE, R^2), fornecendo uma estimativa mais robusta da capacidade de generalização do modelo

  - Quando utilizada em conjunto com técnicas de ajuste de hiperparâmetros (como *Grid Search* ou *Random Search*), a validação cruzada permite comparar diferentes configurações do modelo e selecionar aquela que apresenta melhor desempenho médio



Entendendo e Preparando Dados com Scikit Learn

- Conjunto de Teste (10% a 15% dos dados)
  - Depois de todas as decisões tomadas, como este modelo se comporta em dados nunca vistos?
  - Exemplo:

Idade	Renda	Visitas Site	Comprou
29	3900	11	Sim
47	7100	4	Não
35	5500	15	Sim

    - O resultado desta última execução representa a eficiência do modelo



Entendendo e Preparando Dados com Scikit Learn

- Para que o modelo seja mais robusto e generalize me maneira mais eficiente, é recomendada a utilização dos 3 conjuntos (Treinamento, Validação e Teste) com Validação Cruzada (*Cross Validation*)



Treinamento



Validação



Teste

Treinamento, Validação e Teste com Python

supervisionado_01

```
/Users/ronaldomartinsdacosta/am-env/python_scripts/supervisionado_01.py

...
...
...
```

Entendendo o exemplo construído

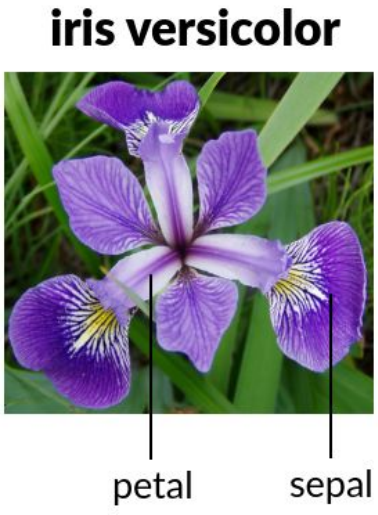

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import (
    train_test_split,
    GridSearchCV
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
iris = load_iris()
X = iris.data
y = iris.target
print("\n================ DATASET ================")
print("Quantidade de amostras:",X.shape[0])
print("Quantidade de caracteristicas:",X.shape[1])
```

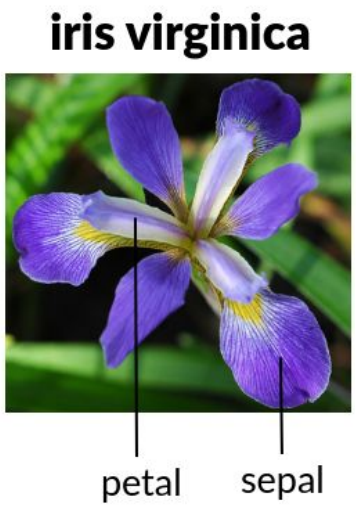
Importa as bibliotecas necessária para execução do exemplo

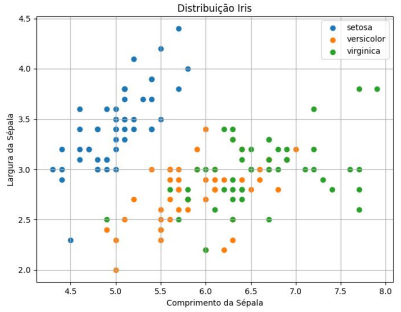
Carrega o exemplo de Íris e mostra em tela as quantidades

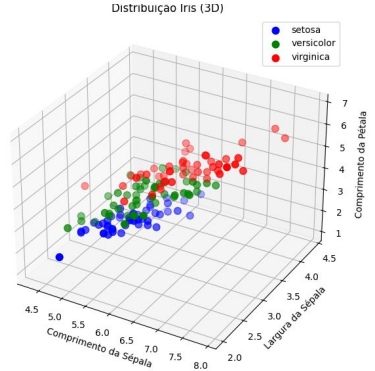
Entendendo o exemplo construído











Entendendo o exemplo construído


```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.15,
    random_state=42,
    stratify=y
)
```

sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)[source]

Aplica a divisão aos dados de entrada
*arrays = Sequência de objetos
test_size = Tamanho do conjunto de teste
train_size = Tamanho do conjunto de treinamento
random_state = Controla o embaralhamento aplicado aos dados antes da divisão
shuffle = Define se os dados devem ser embaralhados antes da divisão
stratify = Realiza uma divisão estratificada dos dados usando os rótulos das classes informados

Entendendo o exemplo construído

supervisionado_01

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.15,
    random_state=42,
    stratify=y
)
```

Tem a função de controlar o processo aleatório de embaralhamento dos dados antes da divisão entre treinamento e teste

Imagine um conjunto formado pelas amostras A B D C E

Sem o random_state

Execução 1:
A B D C E
Execução 2:
C E A B D
Execução 3:
D A E C B

Com o random_state

Execução 1:
D B A E C
Execução 2:
D B A E C
Execução 3:
D B A E C

E POR QUE O VALOR 42 ?

42 ficou famoso porque aparece frequentemente em exemplos
Semelhante ao que ocorre com o nome de variável "i" para definir um valor inteiro
ou "n" para especificarmos as iterações de um laço
mas poderia ser qualquer outro valor: 123, 999, etc.

Entendendo o exemplo construído


```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.15,
    random_state=42,
    stratify=y
)
```

Imagine um conjunto após a divisão:

Treinamento

Classe	Quantidade
A	80
B	35
C	13

Teste

Classe	Quantidade
A	20
B	15
C	**0**

O teste ficou sem exemplos da classe C
O modelo nunca será avaliado nesta classe

o stratify=y garante que isto não acontecerá e que cada subconjunto será proporcional à quantidade de amostras de cada classe existentes no dataset

Entendendo o exemplo construído

supervisionado_01

```
parametros = {
    "knn__n_neighbors": range(1,8),
    "knn__weights":["uniform","distance"],
    "knn__metric":["euclidean","manhattan"],
    "knn__algorithm":["auto","ball_tree","kd_tree"]
}
grid = GridSearchCV(
    estimator=pipeline,
    param_grid=parametros,
    cv=10,
    scoring="accuracy",
    n_jobs=-1,
    return_train_score=True
)
grid.fit(X_train,y_train)
```

class sklearn.model_selection.GridSearchCV(estimator, param_grid, *, scoring=None, n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False)

Entendendo o exemplo construído

class sklearn.model_selection.GridSearchCV(estimator, param_grid, *, scoring=None, n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False)

estimator = Implementa a interface de estimador

param_grid = Dicionário contendo os nomes dos parâmetros

scoring = Estratégia utilizada para avaliar o desempenho do modelo com validação cruzada no conjunto de teste

n_jobs = Número de tarefas que serão executadas em paralelo (-1 utiliza todos os processadores disponíveis)

refit = Treina novamente um estimador utilizando os melhores parâmetros encontrados em todo o conjunto de dados

cv = Quantidade de conjuntos para realizar o *Cross Validation*

verbose = Nível de detalhamento das informações exibidas durante a execução

pre_dispatch = Número de tarefas que são enviadas durante a execução paralela

error_score = Valor atribuído à pontuação caso ocorra um erro durante a execução

return_train_score = False não incluirá as pontuações de treinamento

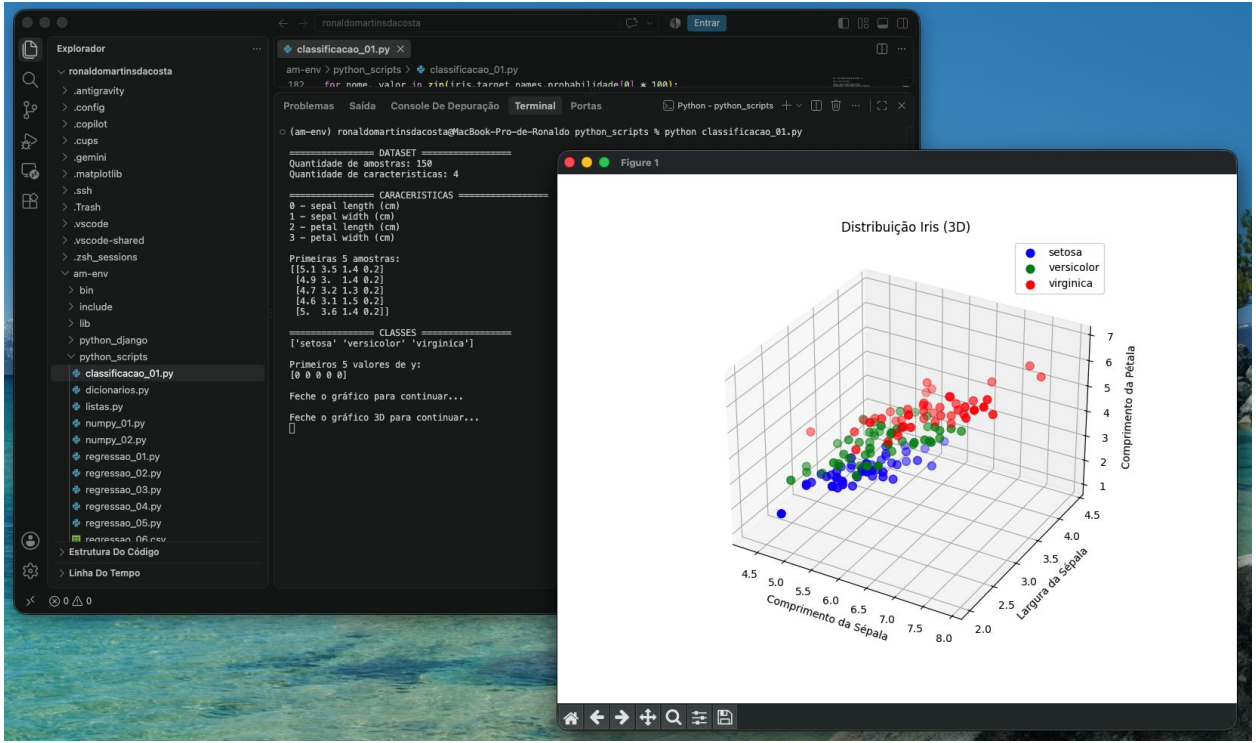
Entendendo o exemplo construído


```
parametros = {
    "knn__n_neighbors": range(1,8),
    "knn__weights":["uniform","distance"],
    "knn__metric":["euclidean","manhattan"],
    "knn__algorithm":["auto","ball_tree","kd_tree"]
}
grid = GridSearchCV(
    estimator=pipeline,
    param_grid=parametros,
    cv=10,
    scoring="accuracy",
    n_jobs=-1,
    return_train_score=True
)
grid.fit(X_train,y_train)
```

O GridSearchCV irá testar dezenas de configurações diferentes. Para decidir qual delas é a melhor, ele precisa de um critério de avaliação. Esse critério é definido pelo parâmetro scoring.

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

supervisionado_01



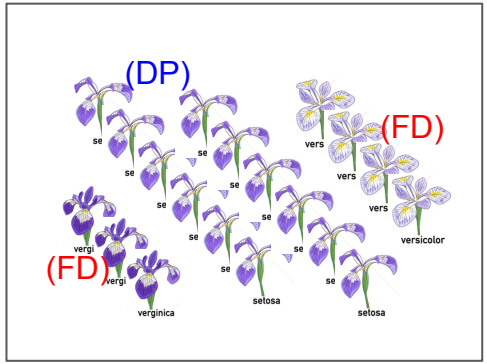


Avaliação de um modelo (**Dados Fictícios**)

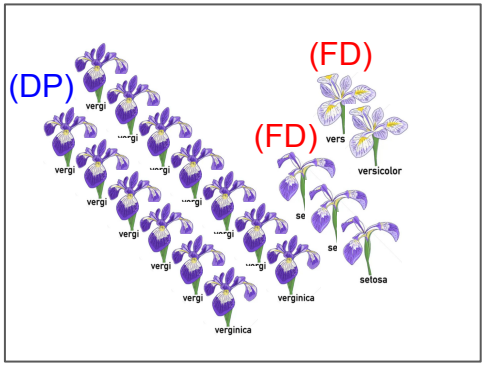


Quando a classificação estiver correta, os elementos compõem a Diagonal Principal (DP) da Matriz de Confusão e quando não estiver correta eles estarão Fora da Diagonal (FD)

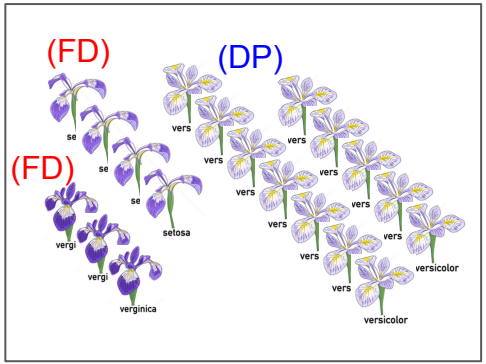
SETOSA



VIRGINICA



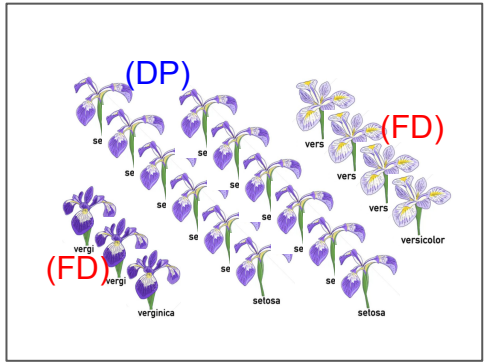
VERSICOLOR



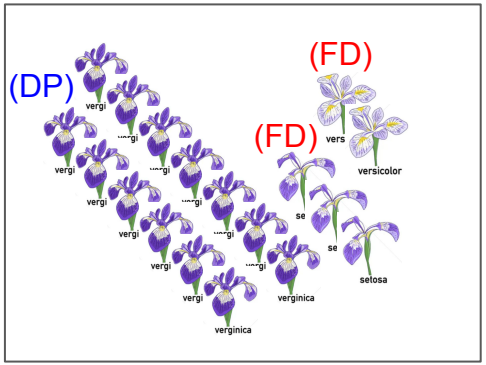
Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	2	6

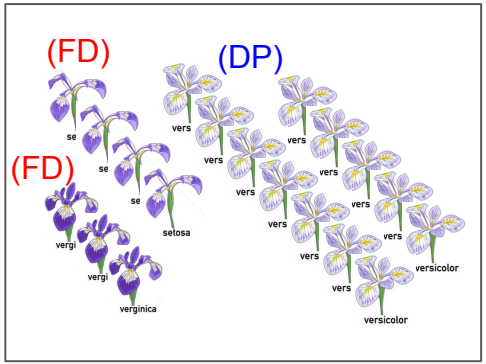
SETOSA



VIRGINICA



VERSICOLOR



Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	2	6

Acurácia da Classificação para o Conjunto de Teste

Total de Elementos do Conjunto de Teste: 7+1+2+3+8+4+5+2+6=38
Total de Elementos Classificados Corretamente: 7+8+6=21
Acurácia do Modelo: 21/38 = 55.26%

VP+FP+FN+VN=7+3+8+27=45

Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	9	6
	Totais	**15**	**20**	**12**

Vamos verificar detalhes por Classe

Verdadeiro Positivo (VP)
Falso Positivo (FP)
Falso Negativo (FN)
Verdadeiro Negativo (VN)

Classe Setosa

VP = 7
FP: 1+2 = 3
FN: 3+5 = 8
VN: 8+9+4+6 = 27

PRECISION

$$\frac{VP}{VP + FP}$$

RECALL

$$\frac{VP}{VP + FN}$$

ESPECIFICIDADE

$$\frac{VN}{VN + FP}$$

ACURÁCIA

$$\frac{VP + VN}{VP + VN + FP + FN}$$

F1-SCORE

$$2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	9	6
	Totais	**15**	**20**	**12**

Vamos verificar detalhes por Classe

Verdadeiro Positivo (VP)

Classe Setosa

VP: 7

VERDADEIRO POSITIVO → Amostras que a previsão do modelo está correta

Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	9	6
	Totais	**15**	**20**	**12**

Vamos verificar detalhes por Classe

Falso Positivo (FP)

Classe Setosa

FP: 1+2 = 3

FALSO POSITIVO → Amostras que a previsão do modelo diz que **são** de uma classe porém **não são**, ao invés disso elas são de outra classe

Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	9	6
	Totais	**15**	**20**	**12**

Vamos verificar detalhes por Classe

Falso Negativo (FN)

Classe Setosa

FN: 3+5 = 8

FALSO NEGATIVO → Amostras que **são** da classe mas o modelo diz que **não são**, o modelo diz que são de outra classe

Avaliação de um modelo (**Dados Fictícios**)

		CLASSE REAL		
		Setosa	**Virgínica**	**Versicolor**
CLASSE PREDITA	**Setosa**	7	1	2
	Virgínica	3	8	4
	Versicolor	5	9	6
	Totais	**15**	**20**	**12**

Vamos verificar detalhes por Classe

Verdadeiro Negativo (VN)

Classe Setosa

VN: 8+9+4+6 = 27

VERDADEIRO NEGATIVO → Amostras que **não são** da classe e o modelo diz que **não são** da classe

Precision

- A *Precision* (precisão) em um problema de classificação mede a proporção das previsões positivas de uma determinada classe que realmente estavam corretas

- Quando o modelo diz que uma amostra pertence a uma classe, com que frequência ele está certo?

- *Preciosion* para a Classe Setosa → 7 / (7 + 3) = 70,00%

$$\text{PRECISION} \quad \frac{VP}{VP + FP}$$

Macro *Precision*

- Calcula a *Precision* de cada classe e faz a média simples

- É útil quando você quer avaliar se o modelo trata bem todas as classes, inclusive classes raras

- Neste caso, todas as classes têm a mesma importância

Weighted Precision

- ➔ Faz a média ponderada pelo número de exemplos de cada classe

- ➔ Representa melhor o desempenho global quando existe desbalanceamento de classes

- ➔ Serão atribuídos pesos para as classes, que podem indicar quais classes são mais importantes

Micro Precision

- Agrupa todos os verdadeiros positivos e falsos positivos
  - Considera o desempenho geral do modelo

- Considerando todas as previsões juntas, qual percentual das previsões positivas estava correto?

$$\text{Precision}_{micro} = \frac{\sum VP}{\sum FP + \sum VP}$$

A importância do *Precision*

➔ A Medida *Precision* (precisão) é importante quando:

- Falsos positivos são caros
  - Detecção de fraude
  - Diagnóstico médico
  - Detecção de conteúdo impróprio

- Se deseja ter confiança nas previsões
  - Um modelo que recomenda medicamentos

Recall

- O *Recall* (também chamado de sensibilidade, taxa de verdadeiros positivos ou *True Positive Rate* - TPR) mede a capacidade do modelo de encontrar corretamente os exemplos de uma determinada classe que realmente existem

- Dentre todos os exemplos que realmente pertencem a uma classe, quantos o modelo conseguiu identificar corretamente?

- *Recall* para a Classe Setosa → 7 / (7 + 8) = 46,66%

$$\text{RECALL} \quad \frac{VP}{VP + FN}$$

Recall por Classe

➔ É o Recall calculado individualmente para cada classe

➔ Neste exemplo teremos:

- *Recall* para a Classe Setosa → 7 / (7 + 8) = 46,66%
- *Recall* para a Classe Virgínica → 8 / (8 + 1 + 9) = 44,44%
- *Recall* para a Classe Versicolor → 6 / (6 + 4 + 2) = 50,00%

Recall Macro

- Calcula o Recall por cada classe e depois tira a média aritmética deles
  - Para o exemplo em questão sería (46.66 + 44.44 + 50.00) / 3 = 47.03%
- Em média, considerando que todas as classes têm a mesma importância, qual percentual dos exemplos de cada classe o modelo consegue encontrar?
  - É importante quando:
    - Todas as classes possuem a mesma relevância
    - Não desejamos que uma classe dominante esconda o desempenho ruim de outra
    - Existe desbalanceamento de classes

Recall Macro

➔ Exemplo:

- Classe (Doença Rara): 50 amostras
- Classe (Pessoas Saudáveis): 50.000 amostras

Mesmo sendo rara, a doença pode ser muito importante.O *Recall* Macro garante que a classe rara tenha o mesmo peso

Recall Weighted

- Calcula o *Recall* de cada classe, mas faz uma média considerando o número de exemplos existentes em cada classe

$$Recall_{weighted} = [(N_1 \times Recall_1) + (N_2 \times Recall_2) + (N_3 \times Recall_3)] / N$$

$N_n \rightarrow$ É o numero de amostras da classe

- O *Recall Weighted* informa qual é o desempenho médio do modelo considerando a quantidade real de exemplos de cada classe?

  - É recomendado quando:
    - Existe desbalanceamento entre classes
    - A proporção real das classes importa
    - Se deseja uma medida mais próxima do desempenho global

Recall Weighted

➔ Exemplo - Classificação de Clientes

- Cliente comuns: 95%
- Cliente premium: 5%

Se a distribuição real importa, use *Recall Weighted*, caso contrário utilize *Recall Macro*

Recall Micro

➔ O *Recall Micro* soma todos os verdadeiros positivos e todos os falsos negativos antes de calcular.

$$Recall_{Micro} = \sum VP / (\sum FN + \sum VP)$$

Como utilizado o *Recall*

- O *Recall* é uma das métricas mais importantes na avaliação de modelos de aprendizado de máquina porque mede a capacidade do modelo de identificar corretamente os exemplos que realmente pertencem a uma determinada classe

- Um modelo pode apresentar uma elevada acurácia e, ainda assim, possuir um *Recall* muito baixo para uma determinada classe

- Um *Recall* elevado indica que:
  - o modelo consegue identificar a maioria dos exemplos pertencentes à classe
  - existem poucos falsos negativos
  - dificilmente um exemplo positivo deixa de ser identificado

Como utilizado o *Recall*

➔ Um *Recall* baixo indica que:

- muitos exemplos da classe estão sendo perdidos
- existe uma quantidade elevada de falsos negativos
- o modelo apresenta dificuldade para reconhecer essa classe

Nesse caso, mesmo que as previsões realizadas sejam corretas, muitos exemplos positivos deixam de ser detectados

Em quais situações o *Recall* é importante?

➔ Quando deixar de identificar um exemplo positivo pode causar consequências graves

- diagnósticos médicos
- detecção de situação de perigo/emergência
- detecção de fraudes e delitos

Como utilizado o *Recall*

Situação	Melhor escolha
Classes equilibradas	Macro ou Micro
Classes desbalanceadas	Macro
Classe rara é importante	Macro
Quero desempenho geral do sistema	Micro
Quero representar a distribuição real dos dados	Weighted
Problemas médicos, fraude, segurança	Normalmente Macro + análise por classe

Precision x *Recall*

- Embora frequentemente analisados em conjunto, *Recall* e *Precision* medem aspectos diferentes

Recall avalia a capacidade do modelo de encontrar todos os exemplos positivos

Precision avalia a confiabilidade das previsões positivas realizadas pelo modelo

- Um modelo pode apresentar:
  - *Recall* alto e *Precision* baixa: identifica quase todos os exemplos positivos, mas gera muitos falsos positivos
  - *Precision* alta e *Recall* baixa: realiza poucas previsões positivas, porém com alta confiabilidade, deixando muitos exemplos positivos sem identificação

Especificidade

- Enquanto o *Recall* (Sensibilidade) mede a capacidade do modelo de identificar corretamente os exemplos positivos, a **Especificidade** mede a capacidade do modelo de identificar corretamente os exemplos negativos

- De todos os exemplos que realmente não pertencem à classe analisada, quantos o modelo conseguiu identificar corretamente como negativos?

- Tomando como exemplo um caso de um classificador para identificar pessoas com uma certa doença
  - A **Especificidade** mede a capacidade do modelo de identificar corretamente os indivíduos que **não possuem a doença**. Em termos percentuais, ela indica que, dentre todas as pessoas realmente saudáveis, o modelo classificou corretamente **X%** como não portadoras da doença.

$$\textbf{ESPECIFICIDADE} \quad \frac{VN}{VN + FP}$$

Especificidade

➔ Para o exemplo de classificação de iris Setosa no Dataset íris:

Especificidade → 27 / (27 + 3) = 90,00%

➔ Das flores que não eram Setosa, o modelo classificou corretamente aproximadamente 90% como "não Setosa"

➔ A Especificidade é particularmente importante quando um falso positivo possui um custo elevado. Enquanto o *Recall* busca reduzir falsos negativos, a Especificidade busca reduzir falsos positivos. Dependendo da aplicação, um falso positivo pode gerar custos significativos

ESPECIFICIDADE

$$\frac{VN}{VN + FP}$$

Especificidade

- Uma Especificidade elevada indica que:
  - o modelo produz poucos falsos positivos
  - dificilmente classifica exemplos negativos como positivos
  - possui boa capacidade de rejeitar exemplos que não pertencem à classe

- Uma Especificidade baixa indica que:
  - muitos exemplos negativos estão sendo classificados como positivos
  - o modelo gera muitos falsos alarmes
  - existe uma quantidade elevada de falsos positivos

$$\textbf{ESPECIFICIDADE} \quad \frac{VN}{VN + FP}$$

Especificidade

- Em problemas multiclasse, a Especificidade é calculada para cada classe individualmente, utilizando a estratégia *One-vs-Rest* (Um contra Todos)
  - O que significa "One-vs-Rest"?
  - Setosa × Não Setosa ou seja:
    - Setosa → Positivo | Não Setosa → Negativo
      Depois repete para as demais classes

- Entre todas as amostras que NÃO pertencem a esta classe, quantas o modelo conseguiu identificar corretamente como NÃO pertencentes a ela?

ESPECIFICIDADE

$$\frac{VN}{VN + FP}$$

Acurácia

- Mede a proporção de previsões que o modelo classificou corretamente em relação ao total de previsões realizadas

- Uma acurácia de 95% significa que a cada 100 previsões o modelo classifica corretamente 95 vezes

- Para a Classe de Iris **Setosa**
  - Acurácia → (7 + 27) / (7 + 27 + 3 + 8) = **75,55%**

$$\textbf{ACURÁCIA} \quad \frac{VP + VN}{VP + VN + FP + FN}$$

Quando utilizar a Acurácia

- As classes são equilibradas
- Os custos dos erros são semelhantes
  - errar uma classe tem aproximadamente o mesmo impacto que errar outra
- Avaliação inicial de modelos
  - Quando estamos comparando os modelos (knn, SVM, Random Forest, etc.)

- **A principal limitação da acurácia aparece quando existe desbalanceamento de classes**

ACURÁCIA

$$\frac{VP + VN}{VP + VN + FP + FN}$$

Tipos de Acurácia

- ➔ Accuracy Macro
  - ◆ calcula a média dos recalls das classes: $Balanced_{Accuracy} = (Recall_1 + Recall_2 + ... + Recall_n) / n$
  - ◆ evita que uma classe dominante esconda o desempenho ruim de classes menores

- ➔ Accuracy Weighted
  - ◆ considera pesos diferentes para cada classe
  - ◆ não é recomendada para avaliação multiclasses por gerar distorções

- ➔ Accuracy Micro
  - ◆ considera todas as previsões juntas, sem separar por classe

$$Accuracy_{micro} = \sum acertos / \sum amostras$$

ACURÁCIA

$$\frac{VP + VN}{VP + VN + FP + FN}$$

Tipos de Acurácia

Métrica	Como calcular	Deve usar?
Accuracy tradicional	Acertos totais / total de amostras	Sim, quando classes são equilibradas
Accuracy Micro	Igual à Accuracy em multiclasses simples	Não agrega informação nova
Accuracy Weighted	Média ponderada de acurácias por classe	Geralmente não recomendada

Tipos de Acurácia

- *Acurácia Micro* é praticamente a mesma coisa que a acurácia tradicional em classificação multiclasse de rótulo único

- *Acurácia Weighted* não é uma métrica amplamente utilizada porque a acurácia já incorpora naturalmente a distribuição das classes e a versão ponderada pode ser enganosa

- Para problemas multiclasses, principalmente com desbalanceamento, é mais recomendado utilizar *Balanced Accuracy*, *Recall Macro*, *F1 Macro* e métricas por classe

F1-*Score*

- Combina a *Precision* e o *Recall* em uma única medida. Ele é especialmente útil quando não basta saber apenas quantos exemplos o modelo encontrou (*Recall*) ou quão confiáveis são suas previsões positivas (*Precision*)

- Imagine dois modelos para identificar a classe Setosa.
  - Modelo A
    - Precision = 95%
    - Recall = 40%
    - **O modelo quase nunca erra quando diz que uma flor é Setosa, mas deixa escapar muitas Setosas**
  - Modelo B
    - Precision = 55%
    - Recall = 98%
    - **O modelo encontra praticamente todas as Setosas, mas classifica muitas flores de outras espécies como Setosa**

F1-SCORE

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1-*Score*

- Qual modelo é melhor?
  - Apenas olhando *Precision* ou *Recall*, não é possível responder

- F1-*Score* para a classe **Setosa**
  - F1 → 2 x [(70.00 x 46.66) / (70.00 + 46.66)] = **55.99%**

- **Setosa**
  - Recall → 70.00
  - Precision → 46.66
  - F1 → 55.99 (um valor entre o *Recall* e *Precision*, equilibrando a avaliação)

F1-SCORE

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Tipos de F1-*Score*

- Por Classe
  - Calcula-se um F1 para cada classe
  - É o mais informativo e mostra exatamente quais classes estão sendo bem classificadas
  - É o apresentado pelo Scikit-Learn na classification_report
- F1 Macro
  - Calcula a média simples dos F1 individuais
  - Quando utilizar:
    - Quando as Classes estiverem balanceadas
    - Todas as classes possuem a mesma importância
    - Avaliação justa de classes raras
- F1 *Weighted*
  - Cada classe recebe um peso proporcional à quantidade de exemplos reais.
  - Classes desbalanceadas
  - Quando utilizar:
    - Quando as Classes estiverem Desbalanceadas
    - Quando se deseja representar o desempenho proporcional ao tamanho de cada classe
- F1 Micro
  - Somam-se todos os VP, FP e FN

Tipos de F1-*Score*

Tipo	Como é calculado	Quando utilizar
F1 por classe	Calculado individualmente para cada classe	Para identificar quais classes apresentam melhor ou pior desempenho
F1 Macro	Média simples dos F1 das classes	Quando todas as classes têm a mesma importância, mesmo que sejam desbalanceadas
F1 Weighted	Média ponderada pelo número de exemplos de cada classe	Quando se deseja refletir a distribuição real das classes no conjunto de dados
F1 Micro	Calculado a partir da soma global de VP, FP e FN	Quando se deseja avaliar o desempenho global do modelo considerando todas as amostras igualmente

Tipos de F1-*Score*

Situação	F1 recomendado
Desejo analisar cada classe individualmente	F1 por classe
Classes igualmente importantes	F1 Macro
Classes desbalanceadas e deseja-se refletir sua frequência	F1 Weighted
Interesse no desempenho global do classificador	F1 Micro

F1-*Score*

➔ O F1-Score é uma métrica de equilíbrio entre *Precision* e *Recall*

- Útil quando se deseja um modelo que seja ao mesmo tempo preciso (poucos falsos positivos) e sensível (poucos falsos negativos)

- Problemas multiclasse, a análise mais completa consiste em observar o F1 de cada classe e complementar essa avaliação com os valores Macro, Weighted e Micro

Métricas de Avaliação de um Modelo

Métrica	Por Classe	Macro	Weighted	Micro
Recall	✅	✅	✅	✅
Precision	✅	✅	✅	✅
Especificidade	✅	✅	✅	⚠️ menos utilizada
Acurácia	⚠️ abordagem One-vs-Rest	⚠️ não usual	⚠️ não usual	Acurácia
F1-Score	✅	✅	✅	✅

Universidade Federal de Goiás
Instituto de Informática
Prof. Dr. Ronaldo Martins da Costa

UFG
INF
INSTITUTO DE INFORMÁTICA

